GRWM: CUDA Mesh Preprocessing Pipeline

Geometry Resampling and Weighting for Meshes

A Parallel GPU Preprocessing Tool for Procedural Geometry Systems

Michael Salton

Companion project to Gravel

ABSTRACT

This document describes the design, implementation, and evaluation of a CUDA-based mesh preprocessing pipeline developed as a companion tool to Gravel, a real-time procedural resurfacing system built on Vulkan mesh shaders. The pipeline accepts a raw triangle mesh and produces three binary buffer files consumed directly by Gravel at load time: per-vertex mean curvature values, per-face feature edge flags, and per-face priority-sorted element placement grids. Without these buffers, Gravel operates as a straight reimplementation of Raad et al. with uniform element placement and no temporal stability. With them, the task shader gains the data it needs to concentrate element density at high-curvature regions and silhouettes, and to assign stable priority-ordered positions to elements so that LOD transitions produce smooth individual appearance and disappearance rather than global position reorganization. All three stages are implemented as CUDA kernels and benchmarked against single-threaded CPU reference implementations. The project demonstrates parallel sparse linear algebra via cuSPARSE, irregular gather patterns for one-ring neighborhood traversal, atomic accumulation for cotangent weight assembly, and segmented sorting using CUB. The output of this tool directly enables two of Gravel's three novel contributions: curvature-aware element placement and temporally stable LOD transitions.

1. MOTIVATION AND CONTEXT

1.1 The Role of Preprocessing in Gravel

Gravel generates procedural geometry on the GPU at runtime using the Vulkan mesh shader pipeline. For every frame, a task shader decides how many mesh shader invocations to emit per face, and a mesh shader generates the actual geometry for each element. To make these decisions intelligently, the task shader needs three pieces of data that are not present in a raw OBJ file:

Per-vertex mean curvature: used to modulate element density so that high-curvature regions receive more elements than flat regions.

Per-face feature edge flags: used to ensure that faces adjacent to sharp creases always receive full element coverage regardless of curvature magnitude.

Per-face slot priority lists: used to assign stable positions to elements so that as the LOD level changes, elements appear and disappear individually at fixed positions rather than all positions reorganizing simultaneously.

None of this data changes at runtime. It depends only on the base mesh geometry, not on camera position, lighting, or animation state. It is therefore computed once at load time and uploaded to GPU storage buffers that the shaders read from throughout the session.

1.2 Why CUDA

These computations have natural parallel structure. A mesh with 500,000 vertices requires 500,000 independent curvature estimates. A mesh with 1,000,000 edges requires 1,000,000 independent dihedral angle comparisons. A mesh with 200,000 faces with 64 slots each requires 12,800,000 independent priority evaluations followed by 200,000 independent sorts. A single-threaded CPU processes these sequentially. A GPU processes them in parallel across thousands of cores simultaneously.

The three stages also cover a range of GPU programming patterns that are core to parallel computing: sparse matrix-vector multiplication for curvature, embarrassingly parallel map operations for feature edges, and segmented sorting for slot priorities. This makes the pipeline a strong demonstration of CUDA programming techniques beyond simple embarrassingly parallel workloads.

1.3 Independence from Gravel

The CUDA preprocessing tool is a standalone command-line program. It compiles and runs without Gravel, without Vulkan, and without any graphics API dependency. Its only inputs are a standard OBJ mesh file and a set of parameters passed as command-line arguments. Its outputs are binary files in a documented format. Gravel loads these files at startup if they are present, falling back to CPU computation if they are not. The tool can be fully demonstrated, validated, and benchmarked in isolation.

2. SYSTEM OVERVIEW

2.1 Pipeline Stages

The tool runs three sequential stages:

Stage 1 computes per-vertex mean curvature using the cotangent Laplacian. Input is the vertex position array and face index array. Output is a float buffer of length V.

Stage 2 detects feature edges by computing dihedral angles between adjacent face normals. Input is the face normal array and edge adjacency structure. Output is a uint8 buffer of length F indicating whether each face has at least one adjacent feature edge.

Stage 3 computes per-face element slot positions and priorities, then sorts the slots within each face by priority. Input is the face array and the curvature buffer from Stage 1. Output is a structured buffer of length F * N_max containing sorted slot entries.

2.2 Output Binary Format

All three stages write to a single output directory. The file format is intentionally simple: a fixed header followed by raw data. Both the CUDA tool and Gravel's loader include the same header definition:

```
struct PreprocessHeader {
```

```
uint32_t magic;          // 0x47525650 ("GRVP")
uint32_t version;        // 1
uint32_t vertex_count;
uint32_t face_count;
uint32_t edge_count;
uint32_t slots_per_face;
uint32_t padding[2];
};
```

File layout for curvature.bin:

```
PreprocessHeader (32 bytes)
float curvature[vertex_count]
```

File layout for features.bin:

```
PreprocessHeader (32 bytes)
uint8_t feature_flag[face_count] (1 = face has adjacent feature edge)
```

File layout for slots.bin:

```
PreprocessHeader (32 bytes)
SlotEntry slots[face_count * slots_per_face]
where SlotEntry = { float u; float v; float priority; uint32_t slot_index; }
```

2.3 Build and Dependencies

The tool is a standalone CMake project with the following dependencies:

```
Language:        C++17 (host), CUDA 12+
Build system:    CMake 3.20+
CUDA libraries:  cuSPARSE, cuRAND, CUB (included with CUDA toolkit)
Mesh loading:    tinyobjloader (header-only, included)
No graphics API dependency of any kind.
```

3. STAGE 1: PER-VERTEX MEAN CURVATURE

3.1 Mathematical Background

Mean curvature at a vertex v of a triangle mesh is approximated using the cotangent Laplacian, a standard discrete differential geometry operator. For vertex v with one-ring neighbors v_j, mean curvature is:

```
H(v) = (1 / 2A) * sum_j ( (cot a_ij + cot b_ij) * (v - v_j) )
```

where A is the mixed Voronoi area around v, and a_ij and b_ij are the angles in the two triangles sharing the edge (v, v_j) that are opposite to that edge. The scalar mean curvature magnitude is |H(v)|.

This operator is equivalent to a sparse matrix-vector product L * P where L is the cotangent weight matrix and P is the vertex position matrix. L is sparse because each vertex is only connected to its one-ring neighbors, so each row of L has at most valence nonzero entries (typically 6 to 8 for well-conditioned meshes).

3.2 Implementation Approach

The computation is split into two CUDA kernels.

Kernel 1: BuildCotangentWeights

One thread per triangle face. Each thread computes the three cotangent weights for the three edges of its face and atomically accumulates them into the sparse weight matrix stored in COO (coordinate) format. Atomic adds are required because multiple faces share the same edges and would otherwise race.

```
__global__ void BuildCotangentWeights(
const float3* positions,
const uint3*  faces,
float*        coo_values,
uint32_t*     coo_row,
uint32_t*     coo_col,
uint32_t      face_count)
{
uint32_t fid = blockIdx.x * blockDim.x + threadIdx.x;
if (fid >= face_count) return;

uint3 f = faces[fid];
float3 v0 = positions[f.x];
float3 v1 = positions[f.y];
float3 v2 = positions[f.z];

float cot0 = cotangent(v1, v0, v2); // angle at v0
float cot1 = cotangent(v2, v1, v0); // angle at v1
float cot2 = cotangent(v0, v2, v1); // angle at v2

// accumulate into COO entries for edges (v1,v2), (v0,v2), (v0,v1)
atomicAdd(&coo_values[edgeIndex(f.y, f.z)], cot0);
atomicAdd(&coo_values[edgeIndex(f.x, f.z)], cot1);
atomicAdd(&coo_values[edgeIndex(f.x, f.y)], cot2);
}
```

Kernel 2: SpMV via cuSPARSE

After the COO matrix is built it is converted to CSR format using cuSPARSE's COO-to-CSR conversion, then the sparse matrix-vector product L * P is computed using cusparseSpMM. The result is a float3 per vertex representing the mean curvature vector. A final elementwise kernel computes the magnitude to produce the scalar curvature buffer.

3.3 Voronoi Area Computation

The mixed Voronoi area A used in the normalization is computed in a separate kernel, one thread per face, accumulating one third of each non-obtuse triangle's area to each of its vertices (with corrections for obtuse triangles). This is another atomic accumulation pass.

3.4 Numerical Considerations

Cotangent weights blow up as triangle angles approach 0 or pi (degenerate triangles). The implementation clamps cotangent values to the range [-10, 10] to prevent numerical instability on poor quality meshes.

Users are warned if more than 1 percent of triangles hit the clamp threshold, as this indicates mesh quality issues that may affect curvature accuracy.

3.5 Validation

The output is validated against two analytical ground truth cases:

Sphere of radius R: mean curvature should be uniformly 1/R. The test passes if the mean absolute error across all vertices is below 5 percent of 1/R.

Cylinder of radius R: curved surface curvature should be 1/(2R), flat cap curvature should be near zero. Tested separately for each region.

Validation is run automatically when the tool is invoked with the --validate flag.

3.6 Independent Visualization

Without Gravel, curvature output can be visualized by writing a PLY file with per-vertex scalar color mapped from the curvature buffer (blue for low, red for high). MeshLab and Blender both display per-vertex colored PLY files directly. The tool writes this visualization file when invoked with --vis-curvature.

4. STAGE 2: FEATURE EDGE DETECTION

4.1 Definition

A feature edge is an edge where the dihedral angle between the normals of its two adjacent faces exceeds a user-defined threshold. The default threshold is 30 degrees, configurable via --feature-threshold. A face is flagged as a feature face if any of its edges is a feature edge.

4.2 Data Structure: Half-Edge Adjacency

To find the two faces adjacent to each edge, the tool builds a half-edge adjacency structure on the CPU before launching the CUDA kernel. This is a standard mesh data structure where each directed half-edge stores the index of its opposite half-edge (and thus its adjacent face). Building this structure takes O(E) time on the CPU and is fast enough that it does not need to be parallelized.

The half-edge table is uploaded to the GPU as a flat uint32 array before the detection kernel launches.

4.3 CUDA Kernel

One thread per edge. Each thread looks up its two adjacent face indices from the half-edge table, reads their precomputed normals, computes the dot product, converts to an angle, and writes a 1 to a per-edge flag array if the angle exceeds the threshold.

```
__global__ void DetectFeatureEdges(
const float3*   face_normals,
const uint32_t* half_edge_opposite,
const uint32_t* edge_to_face,
uint8_t*        edge_flags,
float           cos_threshold,
uint32_t        edge_count)
{
uint32_t eid = blockIdx.x * blockDim.x + threadIdx.x;
if (eid >= edge_count) return;

uint32_t f0 = edge_to_face[eid];
uint32_t f1 = edge_to_face[half_edge_opposite[eid]];
if (f1 == UINT32_MAX) { edge_flags[eid] = 0; return; } // boundary edge

float d = dot(face_normals[f0], face_normals[f1]);
edge_flags[eid] = (d < cos_threshold) ? 1 : 0;
}
```

A second kernel reduces edge flags to face flags: one thread per face, reads its three edge flags, writes 1 to the face flag if any edge flag is 1.

4.4 Face Normal Precomputation

Face normals are computed in a separate kernel before stage 2, one thread per face, using the cross product of two edge vectors. This kernel is also used in stage 3 and is computed once and reused.

4.5 Validation and Visualization

A cube mesh is used as a validation case: all 12 edges should be flagged as feature edges. A subdivided sphere should have zero feature edges below the threshold.

For visualization, the tool writes the flagged edges as a second OBJ file containing only the feature edge line segments, viewable as a wireframe overlay in any 3D viewer.

5. STAGE 3: SLOT PRIORITY COMPUTATION AND SEGMENTED SORT

5.1 Overview

For each face in the mesh, the system maintains a fixed set of N_max candidate element positions called slots. These slots have positions that never change regardless of LOD level. At runtime, the task shader activates only the top K slots for the current LOD, where K varies with distance. Because slot positions are stable and only activation changes, elements appear and disappear at fixed locations rather than all positions reorganizing.

Stage 3 computes the 2D position and priority score for every slot across every face, then sorts the slots within each face by descending priority. The output is a flat buffer of F * N_max slot entries in sorted order.

5.2 Slot Position Generation

Slot positions are generated by a deterministic hash function of face ID and slot index. This ensures positions are stable across runs and require no storage on the CPU side — they can be recomputed identically at any time.

```
__device__ vec2 slotPosition(uint32_t face_id, uint32_t slot_index) {
uint32_t h = murmurHash(face_id ^ (slot_index * 2654435761u));
return make_float2(
float(h & 0xFFFF) / 65535.0f,
float(h >> 16)    / 65535.0f
);
}
```

The Murmur hash function provides good distribution with low collision rate and is fast to evaluate inside a kernel.

5.3 Priority Score Computation

Each slot's priority is a weighted combination of three factors:

Distance from face center: slots closer to the center of the face have higher priority. This ensures that at low LOD levels, the visible elements are centrally placed and the face does not look empty from a distance.

```
float centerScore = 1.0f - length(pos - make_float2(0.5f, 0.5f)) * 1.414f;
```

Curvature at slot position: the curvature value is bilinearly interpolated from the three vertex curvature values of the face at the slot's barycentric coordinates. Slots in high-curvature regions have higher priority.

```
float curvScore = barycentricInterp(curvature[f.x], curvature[f.y],
curvature[f.z], bary);
```

Jitter: a small random offset breaks ties and prevents visible priority patterns from emerging across adjacent faces.

```
float jitter = fract(sinf(float(slot_index) * 127.1f) * 43758.5f);
```

Final priority:

```
float priority = 0.5f * centerScore + 0.4f * curvScore + 0.1f * jitter;
```

The weights 0.5, 0.4, and 0.1 are exposed as tunable parameters.

5.4 CUDA Kernel: Priority Computation

One thread per slot. Each thread computes its face ID (threadIdx / N_max) and slot index (threadIdx % N_max), generates the slot position, evaluates priority, and writes to the output slot buffer.

```
__global__ void ComputeSlotPriorities(
const uint3*    faces,
const float*    curvature,
SlotEntry*      slots,
uint32_t        face_count,
uint32_t        slots_per_face,
float           w_center,
float           w_curv,
float           w_jitter)
{
uint32_t tid = blockIdx.x * blockDim.x + threadIdx.x;
if (tid >= face_count * slots_per_face) return;

uint32_t fid = tid / slots_per_face;
uint32_t sid = tid % slots_per_face;
uint3    f   = faces[fid];

float2 pos  = slotPosition(fid, sid);
float3 bary = uvToBarycentrics(pos);

float centerScore = 1.0f - length(pos - make_float2(0.5f, 0.5f)) * 1.414f;
float curvScore   = barycentricInterp(curvature[f.x], curvature[f.y],
curvature[f.z], bary);
float jitter      = fract(sinf(float(sid) * 127.1f) * 43758.5f);

slots[tid].u        = pos.x;
slots[tid].v        = pos.y;
slots[tid].priority = w_center * centerScore
+ w_curv   * curvScore
+ w_jitter * jitter;
slots[tid].slot_index = sid;
}
```

5.5 Segmented Sort Using CUB

After priorities are computed, the slots within each face must be sorted by descending priority. This is a segmented sort: F independent sort operations, each over N_max elements. Naive approaches (one CUDA thread per face running a serial sort) are inefficient.

CUB (CUDA Unbound) provides DeviceSegmentedSort which executes all F sorts in parallel with optimal work distribution across thread blocks. The sort operates on the slots buffer in-place using the priority field as the sort key.

```
cub::DeviceSegmentedSort::SortPairsDescending(
d_temp_storage, temp_storage_bytes,
d_keys_in,    d_keys_out,
d_values_in,  d_values_out,
total_slots,
face_count,
d_offsets,
d_offsets + 1
);
```

The offsets array is a simple arithmetic sequence (0, N_max, 2*N_max, ...) since all segments have equal length, making it trivial to construct.

5.6 Why Segmented Sort Matters

For N_max equal to 64 and F equal to 200,000 faces, the naive approach launches 200,000 sequential sorts of 64 elements, each running on a single thread. CUB's segmented sort distributes this across the full GPU, sorting many segments simultaneously. The theoretical speedup matches the number of concurrent thread blocks the GPU can execute, which for this problem size on a modern GPU is typically 50 to 100 times faster than the naive approach. This is one of the primary performance results of the project.

5.7 Validation

Validation samples 100 random faces from the output and checks that for each face the priority values are in non-increasing order. It also checks that slot positions for a given face ID and slot index always match the deterministic hash function output. Both checks are run with --validate.

6. INTEGRATION WITH GRAVEL

6.1 What Changes in Gravel

The integration requires three changes to Gravel's codebase, all on the CPU/loading side:

First, the mesh loader is extended to check for the three binary files in a preprocessing directory alongside the OBJ. If found, it reads them and uploads to the corresponding SSBOs. If not found, it falls back to CPU computation (or uses uniform placeholder values for curvature and sequential slot order).

Second, three new SSBOs are declared in the Vulkan pipeline and bound to descriptor sets: the curvature buffer at binding 3, the feature flag buffer at binding 4, and the slot buffer at binding 5. These bindings are added to the existing descriptor set layout.

Third, the task shader is updated to read from these buffers as described in the Gravel novel contributions documentation.

6.2 What Does Not Change

The mesh shader, fragment shader, parametric surface generation, LOD calculation, frustum culling, and render loop are entirely unchanged. The preprocessing buffers are read-only inputs to the task shader and have no effect on any other part of the pipeline.

6.3 Workflow

The intended workflow is:

```
# Step 1: preprocess the mesh (run once)
./cuda_preprocess assets/bunny.obj --output assets/bunny_preprocess/ --slots 64

# Step 2: run Gravel (reads preprocessed buffers automatically)
./Gravel --mesh assets/bunny.obj --preprocess assets/bunny_preprocess/
```

If the preprocessing directory does not exist or is missing files, Gravel logs a warning and uses fallback values.

7. PERFORMANCE EVALUATION

7.1 Benchmark Setup

All benchmarks compare the CUDA implementation against a single-threaded C++ reference implementation of each stage. The reference implementation uses the same algorithms and produces byte-identical output (verified by checksum). Benchmarks are run on meshes of increasing face count: 1k, 10k, 50k, 100k, 500k, and 1M faces.

Timing is measured using CUDA events for GPU stages and std::chrono for CPU stages. GPU timing includes all memory transfers (host to device and device to host) to give a realistic total cost. Each measurement is averaged over 10 runs.

7.2 Expected Results

Stage 1 (curvature): The CPU reference is memory-bound for large meshes due to irregular one-ring access patterns. CUDA is also memory-bound but benefits from higher memory bandwidth and the ability to overlap computation with memory fetches. Expected crossover point (where CUDA starts winning) is around 50k vertices. Expected speedup at 1M vertices is 20 to 40 times.

Stage 2 (feature edges): This is embarrassingly parallel and both implementations are fast. Expected speedup is 10 to 20 times at large mesh sizes. The absolute times are small (sub-millisecond for reasonable meshes) so this stage is not the bottleneck.

Stage 3 (slot sort): This is where CUDA wins most decisively. The CPU reference sorts each face sequentially using std::sort. The CUDA segmented sort parallelizes all faces simultaneously. Expected speedup at 200k faces with 64 slots each is 50 to 100 times. This is the primary performance result.

7.3 Memory Usage

The tool reports peak device memory usage for each stage. For a 1M face mesh with 64 slots per face, the slot buffer alone is 1M * 64 * 16 bytes = approximately 1 GB. Users with less than 4 GB VRAM should use --slots 32 or process the mesh in chunks. The tool reports an error and exits cleanly if device memory allocation fails.

8. PROJECT STRUCTURE

The CUDA preprocessing tool is organized as follows:

```
cuda_preprocess/
+-- CMakeLists.txt
+-- include/
|   +-- preprocess.h        (shared header, included by Gravel)
|   +-- half_edge.h         (half-edge data structure)
|   +-- cub_helpers.cuh     (CUB wrappers)
+-- src/
|   +-- main.cpp            (CLI entry point, argument parsing)
|   +-- mesh_loader.cpp     (OBJ loading via tinyobjloader)
|   +-- half_edge.cpp       (CPU half-edge construction)
|   +-- stage1_curvature.cu (cotangent Laplacian kernels)
|   +-- stage2_features.cu  (feature edge detection kernel)
|   +-- stage3_slots.cu     (priority computation and segmented sort)
|   +-- validate.cu         (validation kernels)
|   +-- visualize.cpp       (PLY and OBJ visualization output)
+-- tests/
+-- test_sphere.cpp
+-- test_cube.cpp
+-- benchmark.cpp
```

9. LIMITATIONS AND FUTURE WORK

9.1 Current Limitations

CPU half-edge construction: the half-edge adjacency structure used in stage 2 is built on the CPU. For very large meshes (above 2M faces) this becomes a bottleneck. A parallel GPU half-edge construction pass using atomic operations is a natural extension.

Non-manifold meshes: the half-edge structure assumes a 2-manifold mesh (each edge adjacent to exactly two faces). Non-manifold edges (adjacent to more than two faces) are detected and skipped with a warning, but curvature and feature detection results near non-manifold regions may be unreliable.

Static meshes only: the preprocessed buffers are computed once for a static mesh. Skinned or morphed meshes would require recomputation per frame, which is not currently supported. For animated meshes, the CPU fallback in Gravel is more appropriate.

9.2 Future Directions

Parallel half-edge construction: building the adjacency structure in CUDA to remove the last CPU bottleneck for very large meshes.

Adaptive slot count: rather than a fixed N_max slots per face, allocate more slots to high-curvature faces and fewer to flat faces, reducing total memory usage while concentrating detail where it matters.

Streaming large meshes: process meshes too large to fit in device memory in face-indexed chunks with overlap regions to handle boundary vertices correctly.

10. REFERENCES

[1] Raad et al., "Real-time procedural resurfacing using GPU mesh shader," Computer Graphics Forum 44(2), Eurographics 2025.

[2] Meyer et al., "Discrete Differential-Geometry Operators for Triangulated 2-Manifolds," VisMath 2002. Cotangent Laplacian formulation.

[3] NVIDIA CUB Library Documentation, https://nvlabs.github.io/cub/. DeviceSegmentedSort reference.

[4] NVIDIA cuSPARSE Library Documentation. Sparse matrix-vector multiplication.

[5] Akenine-Moller et al., Real-Time Rendering, 4th edition. Half-edge data structure reference.

[6] Murmur Hash, Austin Appleby. Hash function used for deterministic slot position generation.